WEEK 2 · DAY 5 · OVERVIEW

Case Study

Joining the Tables



Created by **Jeff Lenamon**

ABOUT 55 MIN WITH EXERCISES / 8 SECTIONS / 7 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Problem Definition

Yesterday's files passed every check. Join them into one book and hold it to your numbers: 200 rows, 13 columns, 489,000,000 of face. Give every bond a value at the clean price, its coupon income, and a weight. Then where does the money sit: by sector, by grade (by count and by value, and say which), by rating in credit order? What is our average rating, which the letters could not give you? Which ten issuers carry the most, and what share? What matures first, and who are yesterday's seven flagged bonds? Send one workbook, the book and the answers, and leave nothing behind.

ABOUT 55 MIN WITH EXERCISES / 8 SECTIONS / 7 EXERCISES

Your Route Through Day 5

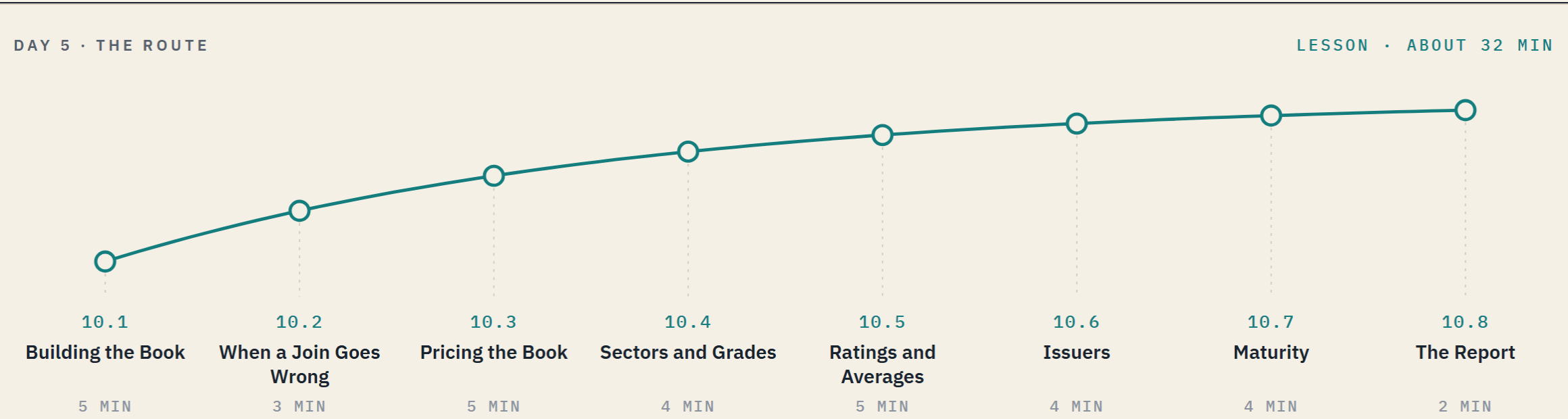


FIGURE 2 Eight sections, in order, with about how long each takes.

Time. About 32 minutes for the lesson, then about 21 for the seven exercises.

10.1 Building the Book

THE FIRST JOIN

```
step_one = pd.merge(positions, issuers, on='ticker', how='left', validate='many_to_one')
assert step_one.shape[0] == rows_before, 'the join changed the row count'
assert step_one['par_held'].sum() == face_before, 'the join changed the face'
print(step_one.shape)
```

```
(200, 10)
```

THE SECOND JOIN

```
book = pd.merge(step_one, ratings, on='rating', how='left', validate='many_to_one')
assert book.shape[0] == rows_before, 'the join changed the row count'
assert book['par_held'].sum() == face_before, 'the join changed the face'
```

200 rows, 13 columns, 489,000,000: yesterday's numbers hold.

10.2 When a Join Goes Wrong

AN OUTER JOIN

```
everything = pd.merge(positions, issuers, on='ticker', how='outer', indicator=True)
print(everything.shape)
print(everything['_merge'].value_counts())
everything[everything['_merge'] == 'right_only'].head(1)
```

```
(235, 11)
_merge
both          200
right_only     35
```

THE ROW CHECK

```
assert rows_after == rows_before, f'expected {rows_before} rows, found {rows_after}'
```

```
AssertionError: expected 200 rows, found 235
```

Left join, then both checks: rows and face.

10.3 Pricing the Book

VALUE

```
book['value'] = book['par_held'] * book['price'] / 100
book[['ticker', 'par_held', 'price', 'value']].head(3)
```

```
ticker  par_held    price      value
0   BEZE   5000000  106.727  5336350.0
1   BEZH    250000   97.556   243890.0
2   BEZL   1500000   98.730  1480950.0
```

COUPON INCOME

```
book['coupon_income'] = book['par_held'] * book['coupon'] / 100
print(book['coupon_income'].iloc[0], book['coupon_income'].sum())
```

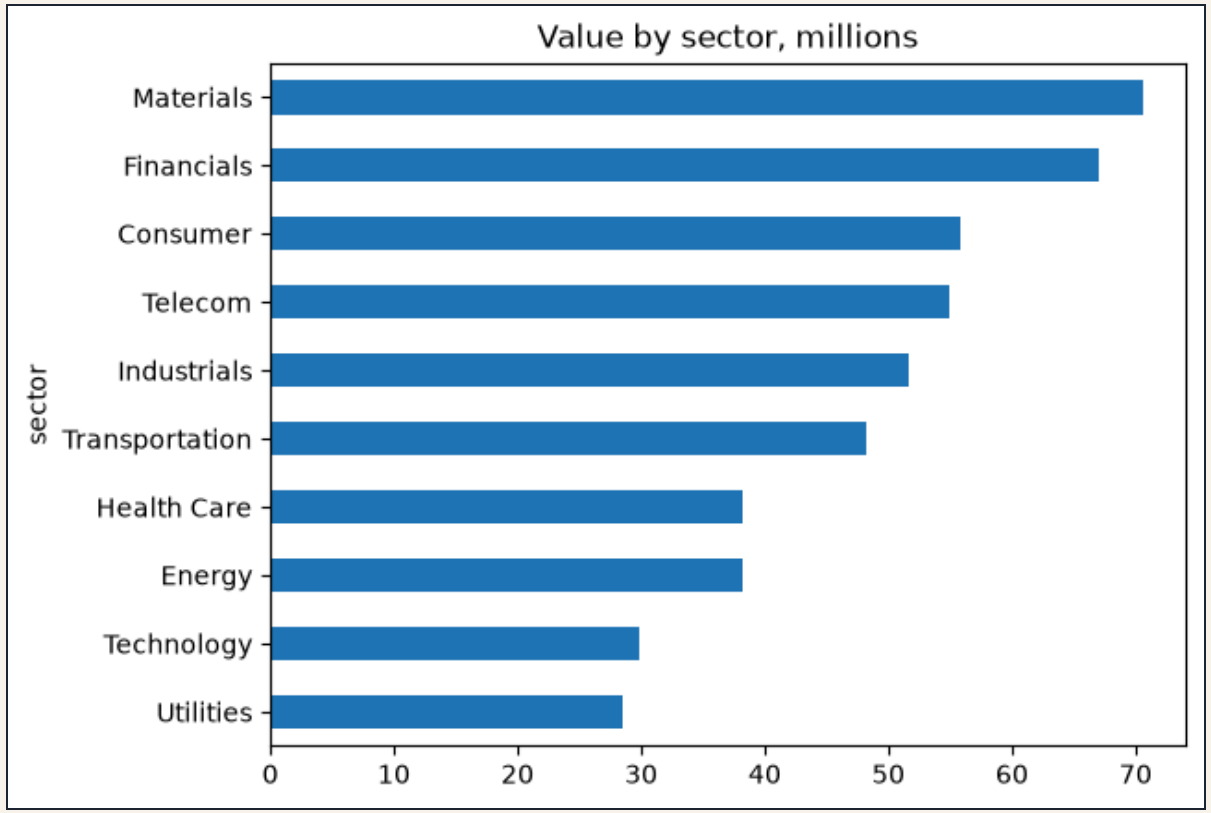
```
350000.0 28864375.0
```

EXCEL SIDE BY SIDE

COLUMN	PANDAS	EXCEL, FILLED DOWN
value	book['par_held'] * book['price'] / 100	=F2 * E2 / 100
coupon_income	book['par_held'] * book['coupon'] / 100	=F2 * C2 / 100
years_to_maturity	(book['maturity'] - as_of).dt.days / 365.25	=(D2 - DATE(2026, 9, 30)) / 365.25
weight_pct	book['value'] / total_value * 100	=N2 / SUM($N$2:$N$201) * 100, the anchors keep the total fixed

Every bond has a value, an income, and a weight that add up: 200 rows, 18 columns.

10.4 Sectors and Grades

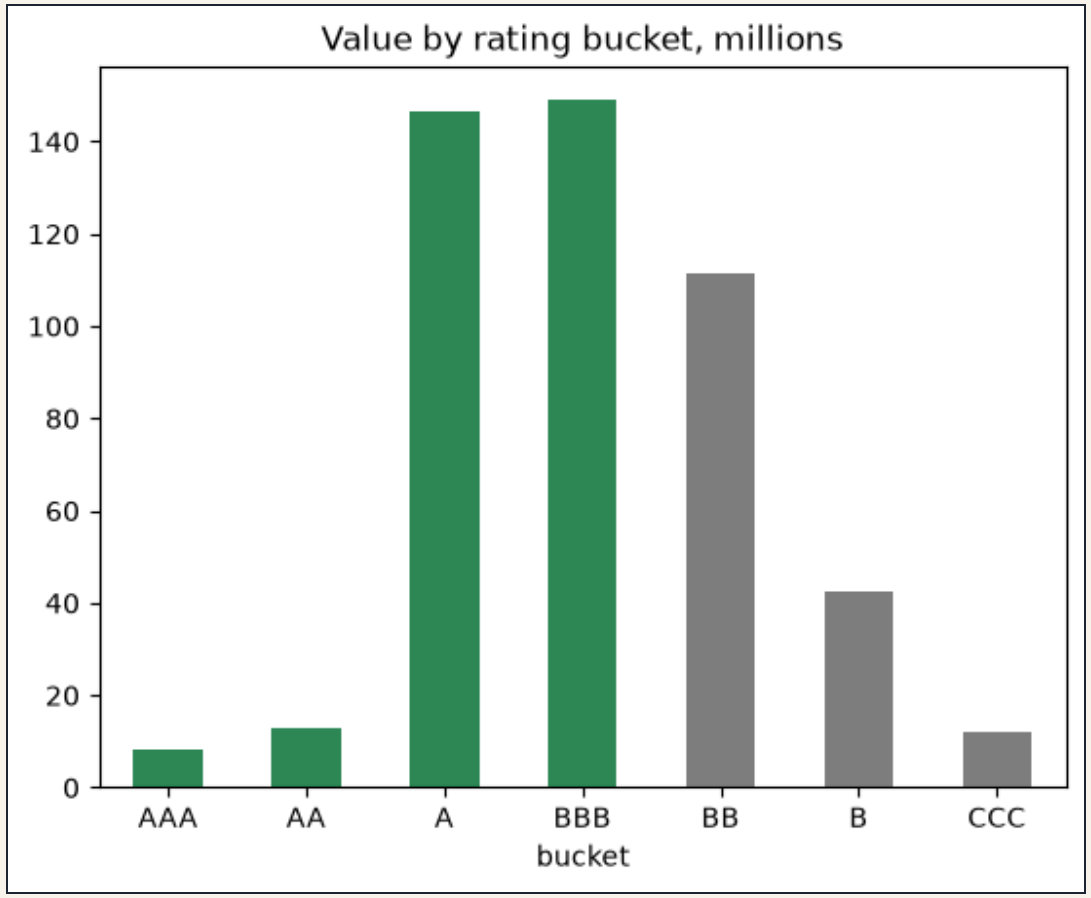


VALUE BY SECTOR

SECTOR	BONDS	VALUE	SHARE_PCT
Materials	26	70527440.0	14.60
Financials	26	67067357.5	13.88
Consumer	26	55773490.0	11.55
Telecom	21	54953580.0	11.38
Industrials	20	51658930.0	10.69
Transportation	20	48159510.0	9.97
Health Care	20	38268482.5	7.92
Energy	17	38258305.0	7.92
Technology	12	29826430.0	6.17
Utilities	12	28530705.0	5.91

Ten sectors add back to 200 bonds and 483,024,230.

10.5 Ratings and Averages



CREDIT ORDER FROM THE SCORE

BUCKET	FIRST_SCORE	BONDS	VALUE	SHARE_PCT
AAA	1	2	8102725.0	1.68
AA	2	8	12928850.0	2.68
A	5	56	146730275.0	30.38
BBB	8	62	148864277.5	30.82
BB	11	49	111511485.0	23.09
B	14	18	42653482.5	8.83
CCC	17	5	12233135.0	2.53

9.25 sits between BBB, score 9, and BBB-, score 10: the average rating yesterday's `TypeError` refused.

10.6 Issuers

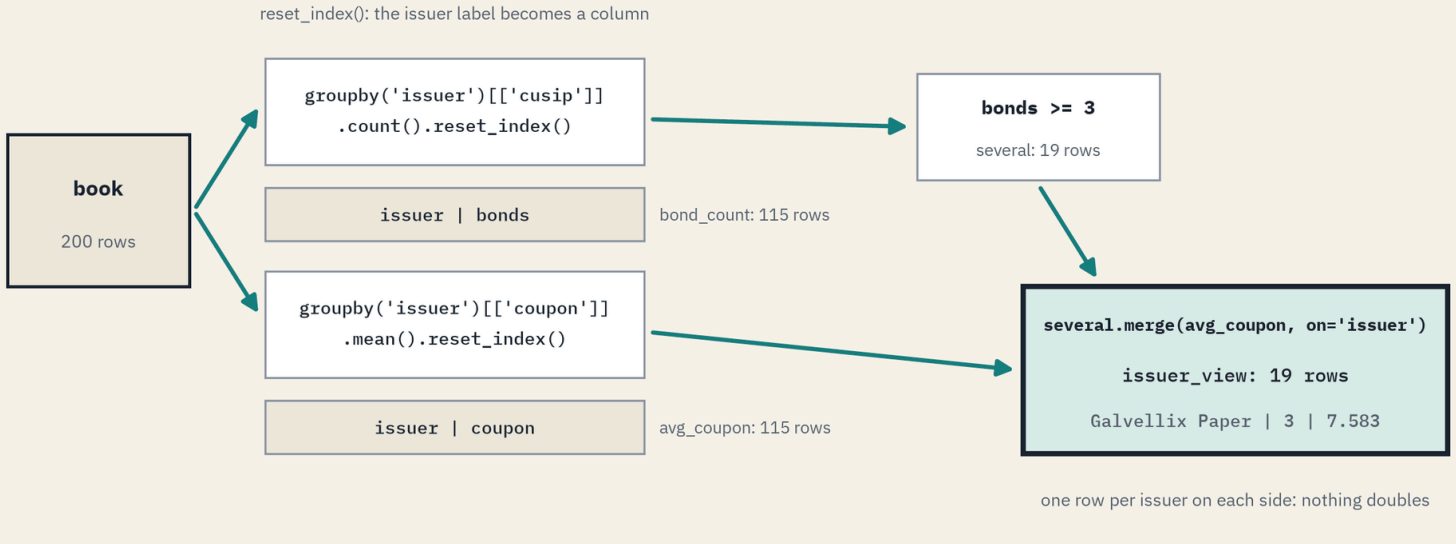


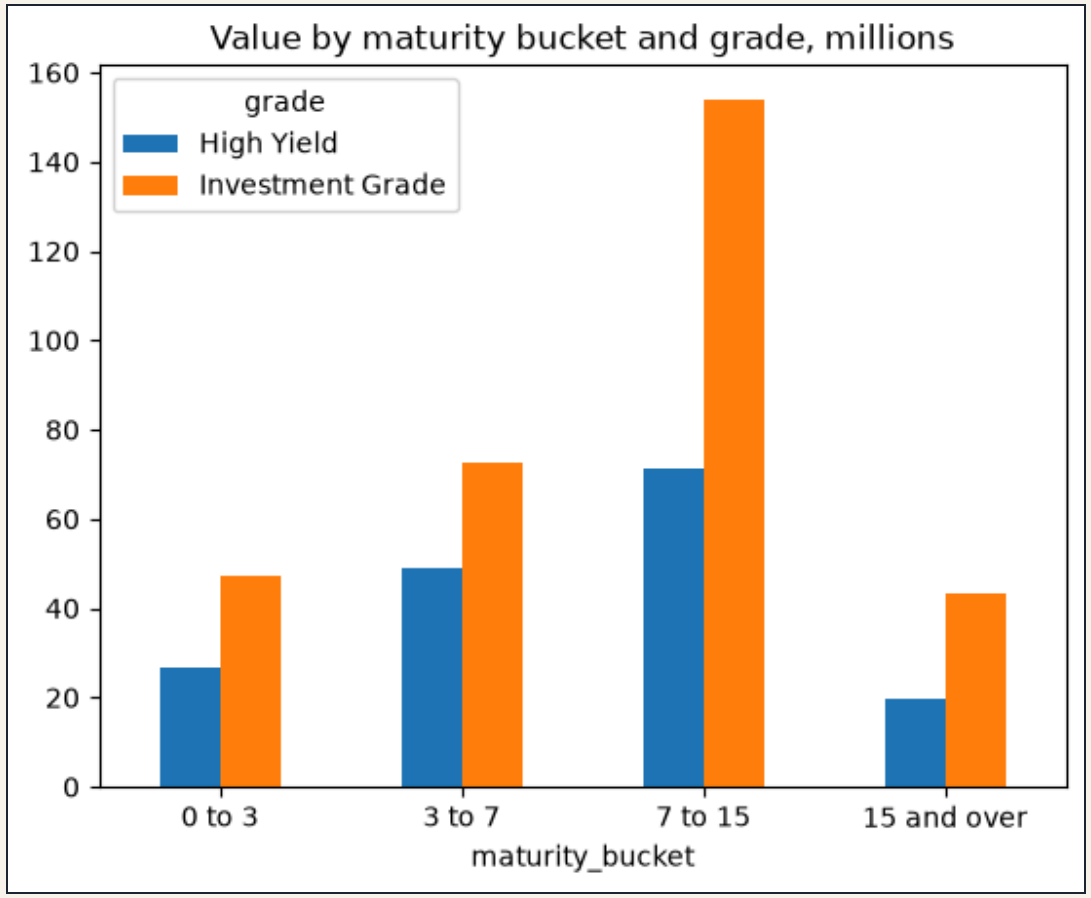
FIGURE 4 Two issuer summaries, reset to columns, filtered to issuers with 3 bonds or more, and merged into 19 rows.

THE MERGE

	ISSUER	BONDS	COUPON
7	Galvellix Paper	3	7.583333
11	Krilellix Networks	3	6.416667
8	Grovostrin Freight	3	6.375000

The join puts a name on every number.

10.7 Maturity



MATURING FIRST

	TICKER	ISSUER	MATURITY
177	VYRH	Veyrholt Health	2027-10-02
112	NEVR	Nevovar Energy	2027-10-24
70	GALX	Galvellix Paper	2027-10-27
125	PLWK	Pellwick Paper	2027-10-28
56	FARX	Farvellix Paper	2027-12-22

28 bonds, 15.29 percent of the book, mature first.

10.8 The Report

WRITE ONE WORKBOOK

```
with pd.ExcelWriter('desk_book.xlsx') as writer:
    book.to_excel(writer, sheet_name='book', index=False)
    by_sector.to_excel(writer, sheet_name='sectors')
    by_grade.to_excel(writer, sheet_name='grades')
    by_bucket.to_excel(writer, sheet_name='buckets')
    top_ten.to_excel(writer, sheet_name='top_ten')
```

READ IT BACK

```
sheets = pd.read_excel('desk_book.xlsx', sheet_name=None, dtype={'cusip': str})
for name, table in sheets.items():
    print(name, table.shape)
```

```
book (200, 18)
sectors (10, 4)
```

The note is answered in one file; next month the same cells run again.

You Can Now

- Join three tables into one book, checked for rows and for face after every join.
- Value a book at the clean price, with weights that add to 100.
- Answer a desk question in one **`groupby`**, and say what each share was divided by.
- Order ratings by their score, and average them through it.
- Group by two columns, **`unstack()`** the result, and merge two summaries.
- Draw bars across with **`plot.barh()`**, stacked or not.

NEXT
WEEK 3 · DAY 1

Visualization

Seven kinds of chart, each drawn from the desk's book, and what to read off each one.